# Reflection on Microsoft's Introduction to Natural Language Processing Concepts: From Statistical Foundations to Semantic Vector Models

Luke Joaquin Bernardo
*College of Computer Studies and Engineering*
*Jose Rizal University*
Taguig City, Philippines
lukejoaquin.bernardo@my.jru.edu

***Abstract*—This paper is a reflection on Microsoft's "Introduction to Natural Language Processing Concepts" module, a short self-paced course covering text analytics, statistical language modeling, and semantic vector representations. Before taking the module, I understood NLP mostly at a surface level, from general AI coursework and things I had read online, rather than from actually working with the techniques myself. The module moved step by step from basic preprocessing, to statistical scoring methods, to embedding-based models, ending with a hands-on exercise comparing a generative model against a specialized statistical tool. This paper covers what I noticed during the hands-on exercise, the key technical ideas I took away, and how these concepts connect to my current coursework and capstone project.**

***Keywords:*** **natural language processing, text analytics, tokenization, TF-IDF, word embeddings, reflection paper**

## I. Introduction

Natural language processing shows up everywhere in modern software, from sorting customer feedback to summarizing documents to powering chatbots. Before starting this module, my understanding of NLP came mostly from using generative AI tools and reading about what they could do, not from actually working through how they do it. I knew, in a general sense, that language models turned text into numbers somehow, but I had not really worked through tokenization, term-weighting math, or the idea of word embeddings.

This module was built to fill in that gap piece by piece [1]. Unit 1 opened with common text analysis tasks, like language detection, entity recognition, and sentiment classification, and Unit 2 moved into tokenization and preprocessing. Unit 3 introduced statistical scoring methods such as term frequency-inverse document frequency (TF-IDF) and graph-based summarization like TextRank, and Unit 4 followed with an introduction to embedding-based semantic models. Unit 5 was a hands-on exercise that let me directly compare a general-purpose generative model with a specialized statistical tool doing the same kinds of tasks, Unit 6 was a short knowledge check, and Unit 7 summarized the module as a whole. This paper walks through what I took from that exercise, summarizes the main technical ideas, and looks at how they connect to my other coursework.

## II. Experiential and Practical Insights

The Unit 5 hands-on exercise was, for me, the part where the earlier theory actually became real. It had two components: a browser-based chat playground running a small generative language model, and a separate "Language Playground" tool built on statistical methods for language detection and personally identifiable information (PII) extraction.

In the chat playground, I asked the model to judge the sentiment of hotel reviews, pull out place names from a promotional announcement, and summarize a multi-speaker meeting transcript, the same three tasks introduced back in Unit 1 as common text analysis use cases. What struck me was how flexible this was: one general-purpose model, with no special setup, handled three different kinds of text analysis just by being told what to do in plain language. This made it clearer to me why generative models have become so popular as general-purpose NLP tools, since they skip most of the pipeline work that older statistical approaches required.

The Language Playground tool gave an important contrast, though. Its language detection and PII extraction features used statistical analysis and pattern matching instead of a generative model, and the documentation was upfront that it would sometimes flag things incorrectly or miss PII entirely. I found that honesty useful, because it pointed to something I had not really thought about before: a smaller, narrower tool can still beat a general model on a well-defined task like PII redaction, where being consistent and predictable matters more than being flexible. The practical lesson I took from this is that choosing between a generative model and a specialized statistical tool is not about which one is "better" overall, but about whether the task calls for flexibility or predictability.

## III. Core Learning Outcomes

Beyond the hands-on exercise, a few technical ideas from the module stood out to me as the real foundation for whatever more advanced NLP topics come later in my coursework.

### A. Tokenization Trade-offs (Unit 2)

Tokenization, or splitting text into pieces that can be analyzed, is the first step in any text analysis pipeline, and it turns out to matter more than I expected. Unit 2 showed that small preprocessing choices can change what an analysis can and cannot detect. Text normalization, for example, helps frequency-based analysis but can also merge words that should stay separate, like a proper name and an ordinary word that happen to look the same. Stemming and lemmatization show a similar trade-off: stemming just chops off word endings using fixed rules and can end up with something that is not even a real word, while lemmatization uses grammar and vocabulary to always land on a valid word, at the cost of more processing. What I took from this is that preprocessing is not just a mechanical first step, it is a design choice that shapes everything that happens afterward.

### B. TF-IDF versus Embeddings (Units 3 and 4)

Comparing the TF-IDF technique from Unit 3 with the embedding-based models from Unit 4 gave me a clear way to think about two different generations of NLP technique. TF-IDF fixes a real problem with plain frequency counting: counting how often a word shows up can describe a single document well enough, but it fails to tell documents apart when the same common words show up everywhere in a collection of documents. TF-IDF fixes this by weighing how often a term appears in one document against how rare it is across the whole collection, so words that appear everywhere end up with little weight, and words that are concentrated in one document stand out.

Embeddings, introduced in Unit 4, solve a different but related problem. TF-IDF and similar statistical measures only look at exact word matches, but embeddings represent words as vectors placed according to their learned meaning, so words with similar meanings end up close together in vector space even if they do not share any letters. The cosine similarity exercises in Unit 4, comparing vectors for words like "dog," "cat," and "tree," showed how closeness in meaning can be measured with geometry instead of just counting shared words. The practical difference, in my view, is that embeddings handle paraphrasing and synonyms much better than statistical counting ever could, no matter how the weighting is tuned.

### C. Context Vectors and Attention (Unit 4)

Unit 4 also briefly touched on contextualized embeddings, the kind produced by attention mechanisms in models like the GPT family, which go a step further than fixed embeddings like Word2Vec or GloVe. A fixed embedding gives a word the same vector no matter what sentence it appears in, while attention-based models calculate a word's vector on the fly, based on the words around it. The unit did not go deep into this, but I saw it as the link between the vector arithmetic covered earlier in Unit 4 and the way the generative model in the Unit 5 exercise seemed to actually understand context. This part made it clearer to me that today's generative language models are really an extension of the embedding ideas from Unit 4, not something completely separate.

Fig. 1 and Fig. 2 show the completion certificate and the Microsoft Learn profile badge earned for finishing all seven units covered in this section, which I am including here as a record of completing this stage of the module.




Fig. 1. Certificate of completion for Microsoft's Introduction to Natural Language Processing Concepts module.

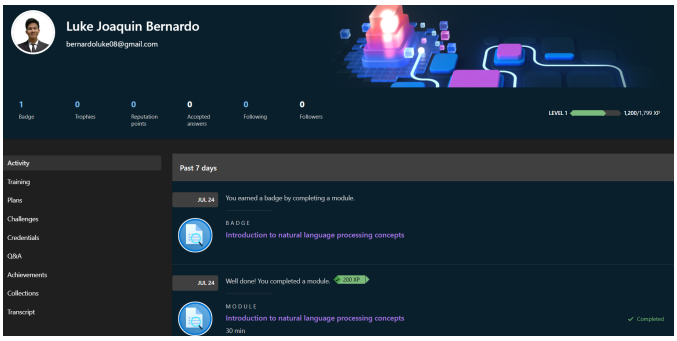



Fig. 2. Microsoft Learn profile showing the badge earned for completing the module.

## IV. Course Application and Impact

The ideas from this module connect directly to both my remaining coursework and my capstone project. My capstone, done with my thesis group under the working title "Integrating NLP for PACUCOA Accreditation Preparedness," uses sentence-transformer embeddings and cosine similarity to help with institutional accreditation document analysis. After our panel defense, we moved our methodology away from a purely predictive analytics framing and toward an NLP-based approach, so the vector-space ideas from this module are not just background knowledge for us, they are things we actually use.

The cosine similarity method from Unit 4, used there to compare document or sentence vectors, is basically the same operation our capstone relies on to match accreditation criteria against institutional documents. The Unit 4 discussion of summarization through sentence-vector centrality, along

with its earlier statistical version from Unit 3, TextRank, is also useful in case our project ends up needing to shorten long accreditation documents into representative excerpts. And the Unit 5 comparison between generative and specialized statistical tools matters at a methodology level too, since we will eventually need to explain why a sentence-transformer similarity approach makes more sense for our specific document-matching task than just prompting a general large language model to do the same comparison.

Outside of the capstone, I expect these same foundations to keep showing up in my other data-oriented coursework, where preprocessing choices like tokenization and normalization will keep determining the quality of whatever analysis comes after, whether it ends up being statistical or neural.

## V. Conclusion and Future Work

This module walked me through text preprocessing in Unit 2, statistical scoring in Unit 3, and semantic vector representation in Unit 4, in that order, and the Unit 5 hands-on exercise gave me an actual basis for comparing generative and specialized tools instead of just reading about the difference. My main takeaways are three things: preprocessing choices like tokenization and normalization shape everything that comes after, no matter what technique is used later; TF-IDF and embedding-based methods solve different problems, one correcting for how often a term shows up across a whole collection of documents, the other capturing meaning beyond exact word matches; and choosing between a generative model and a specialized statistical tool should depend on whether a task needs flexibility or predictability.

Going forward, I want to understand attention mechanisms and transformer architectures more deeply, since this module only covered them at a high level, and they are the basis for the sentence-transformer models my capstone uses. I also want to look into how embedding-based similarity systems are evaluated, such as checking cosine similarity thresholds against human judgments, since this seems necessary to validate our capstone's accreditation-matching approach. More generally, I expect that learning more about large language model fine-tuning and retrieval-augmented generation will build naturally on what this module covered, since both rely on the same embedding and similarity ideas introduced here.

## References

[1] GraemeMalcolm, "Introduction to natural language processing concepts - Training," Microsoft Learn. https://learn.microsoft.com/en-us/training/modules/introduction-language/